

Introduccion a Angular

Arquitectura frontend basada en componentes y comunicacion con APIs

Sandra Cano

Pontificia Universidad Catolica de Valparaiso
OII436-1 Ingenieria Web Avanzada

Segundo semestre de 2026

Proposito de la sesion

Contexto

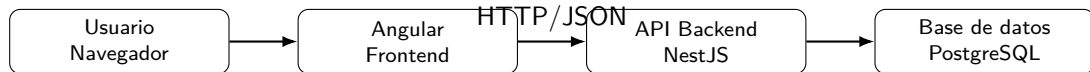
Antes de construir pipelines CI para proyectos con **Angular** y **NestJS**, necesitamos comprender que hace cada tecnologia y como se organiza el frontend.

- Angular se utiliza en la capa de **frontend**.
- Permite construir interfaces web mediante componentes.
- Se comunica con el backend mediante HTTP/JSON.
- Puede validarse con pruebas y build dentro de un pipeline CI.

Pregunta

Si Angular es frontend, **que responsabilidades no deberian quedar en Angular?**

Angular dentro de la arquitectura web



Capa	Responsabilidad principal
Angular	Interfaz, navegacion, formularios, interaccion con el usuario.
NestJS	API, reglas de aplicacion, seguridad, acceso a datos.
PostgreSQL	Persistencia, integridad y consultas.

Que es Angular

Definicion practica

Angular es un **framework frontend** para construir aplicaciones web con TypeScript, componentes, enrutamiento, servicios e integracion con APIs.

Angular ayuda a organizar:

- la interfaz de usuario;
- la navegacion entre vistas;
- la reutilizacion de componentes;
- la comunicacion con servicios backend;
- la validacion de formularios;
- pruebas y construccion del proyecto.

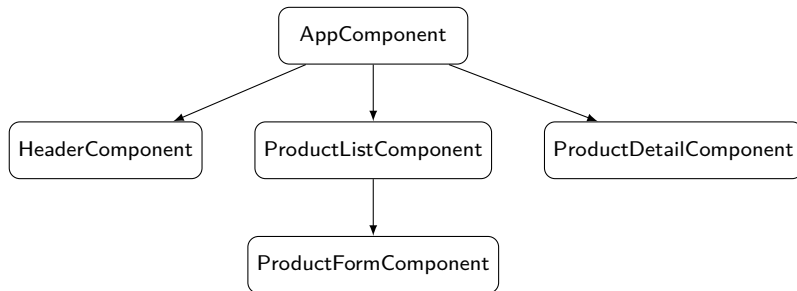
Idea clave

Angular no reemplaza al backend: Angular coordina la experiencia del usuario; el backend protege las reglas criticas y los datos

Tipo de arquitectura en Angular

Arquitectura basada en componentes

Angular organiza la aplicación como un conjunto de **componentes** reutilizables, cada uno con una responsabilidad de interfaz.



Pregunta

Que ventaja tiene dividir una interfaz en componentes pequeños?

Componentes: responsabilidad principal

Un componente debería

- mostrar información;
- reaccionar a eventos del usuario;
- coordinar una vista específica;
- delegar acceso a datos a servicios.

Un componente no debería

- contener reglas críticas del negocio;
- acceder directamente a la base de datos;
- repetir lógica que podría estar en un servicio;
- mezclar demasiadas responsabilidades.

Idea clave

El componente se concentra en la interfaz; el servicio concentra lógica reutilizable y comunicación con APIs.

Anatomia basica de un componente

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-productos',
  standalone: true,
  templateUrl: './productos.html',
  styleUrls: ['./productos.css']
})
export class ProductosComponent {
  titulo = 'Listado de productos';
}
```

Elemento	Funcion
selector	Etiqueta que permite usar el componente en HTML.
templateUrl	Archivo con la vista del componente.
styleUrl	Estilos asociados al componente.
Clase TypeScript	Estado y comportamiento del componente.

¿Qué significa Standalone en Angular?

Standalone Component

Un componente **standalone** puede utilizarse sin necesidad de ser declarado dentro de un NgModule.

Características principales

- Puede importarse directamente en otro componente.
- Declara sus dependencias mediante `imports`.
- Reduce la necesidad de crear módulos sólo para agrupar componentes.
- Favorece una organización más directa y modular.

Angular actual

Los componentes creados actualmente por Angular son **standalone por defecto**.

Navegación con RouterLink

RouterLink

RouterLink es una directiva de Angular que permite navegar entre las rutas de una aplicación sin recargar completamente la página.

Ruta definida

`productos → ProductosComponent`

Enlace en el template

`<a routerLink="/productos">Productos</a>`

Usuario → RouterLink → Angular Router → Componente → RouterOutlet

Idea clave

`routerLink` indica **a qué ruta navegar**; `router-outlet` indica **dónde mostrar el componente asociado a esa ruta**.

Template: vista del componente

```
<h2>{{ titulo }}</h2>

<button (click)="cargarProductos()">
  Cargar productos
</button>

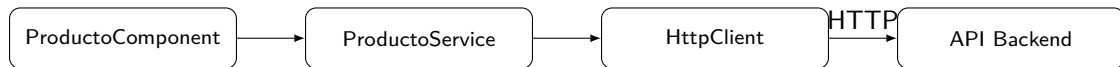
<p>Total: {{ productos.length }}</p>
```

Sintaxis	Significado
titulo	Interpolacion: muestra datos del componente.
(click)	Evento: ejecuta una accion ante interaccion del usuario.
productos.length	Acceso a propiedades del componente.

Servicios e inyeccion de dependencias

Servicio

Un servicio encapsula logica reutilizable: acceso a datos, autentificacion, estado compartido o comunicacion con APIs.



Idea clave

El componente no necesita saber todos los detalles de HTTP; puede delegar esa responsabilidad al servicio.

Ejemplo: servicio que consume una API

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';

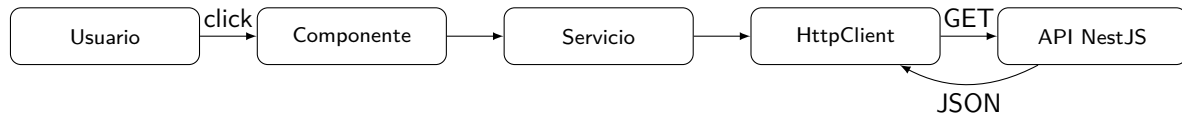
@Injectable({ providedIn: 'root' })
export class ProductoService {
  private http = inject(HttpClient);

  obtenerProductos() {
    return this.http.get('/api/productos');
  }
}
```

Lectura

ProductoService centraliza la comunicacion con el backend. El componente puede llamar al servicio sin conocer la implementacion HTTP.

Flujo de una solicitud desde Angular



- 1 El usuario interactúa con la interfaz.
- 2 El componente invoca un servicio.
- 3 El servicio usa HttpClient.
- 4 El backend responde con datos JSON.
- 5 Angular actualiza la vista.

Router: navegacion entre vistas

Router

El router permite que una aplicacion Angular tenga multiples vistas o paginas dentro de una aplicacion frontend.



Ruta	Componente asociado
/productos	Lista de productos.
/productos/15	Detalle de un producto.
/login	Inicio de sesion.

Ejemplo: configuracion de rutas

```
import { Routes } from '@angular/router';
import { ProductosComponent } from '../productos/productos.component';
import { LoginComponent } from '../login/login.component';

export const routes: Routes = [
  { path: 'productos', component: ProductosComponent },
  { path: 'login', component: LoginComponent },
  { path: '', redirectTo: 'productos', pathMatch: 'full' }
];
```

Idea clave

El router conecta URLs con componentes. Esto permite organizar la aplicacion como una SPA con multiples vistas.

Estructura sugerida del proyecto Angular

Carpeta	Uso sugerido
components/	Componentes reutilizables.
pages/	Vistas principales de la aplicacion.
services/	Logica reutilizable y acceso a APIs.
models/	Interfaces y tipos TypeScript.
shared/	Elementos compartidos.
app.routes.ts	Rutas de la aplicacion.

Ejemplo

src/app/products/product-list/ puede contener el componente que lista productos.

Crear un proyecto Angular

Opcion recomendada para el curso

Fijar una version del CLI ayuda a que todos trabajen con el mismo entorno.

```
npx @angular/cli@20 new frontend --routing --style=css
cd frontend
ng serve --open
```

Comando	Funcion
ng new	Crea un nuevo proyecto Angular.
-routing	Agrega configuracion inicial de rutas.
-style=css	Define CSS como formato de estilos.
ng serve	Ejecuta el servidor de desarrollo.

Crear componentes y servicios

```
# Crear un componente  
ng generate component pages/productos  
  
# Forma abreviada  
ng g c pages/productos  
  
# Crear un servicio  
ng generate service services/productos  
  
# Forma abreviada  
ng g s services/productos
```

Idea clave

El CLI genera archivos base y reduce errores de estructura. Luego el equipo completa la logica de cada componente o servicio.

Formularios en Angular

Para que sirven

Los formularios permiten capturar datos del usuario, validar entradas y enviar informacion al backend.

Template-driven

- Mas simple para formularios pequenos.
- Se apoya mas en el template HTML.

Reactive forms

- Mas explicito y controlado.
- Recomendable para formularios complejos.

Pregunta

Que validaciones deberian estar tambien en el backend, aunque existan en Angular?

Scripts importantes en package.json

```
{
  "scripts": {
    "start": "ng serve",
    "build": "ng build",
    "test": "ng test"
  }
}
```

Comando	Uso
npm start	Ejecuta el servidor de desarrollo.
npm test	Ejecuta el script de pruebas configurado.
npm run build	Construye la aplicacion para distribucion.

Idea clave

En CI se ejecutan comandos definidos por el proyecto. Por eso el archivo package.json es parte central del pipeline.

Por que probar el frontend

Las pruebas permiten verificar que componentes y servicios se comporten como se espera antes de integrar cambios al repositorio principal.

- **Prueba de componente:** verifica que una vista o componente se cree y responda correctamente.
- **Prueba de servicio:** verifica logica reutilizable o acceso a datos simulado.
- **Prueba de integracion:** valida interaccion entre partes del frontend o con un backend simulado.

Pregunta

Si una prueba de Angular falla en un Pull Request, deberia permitirse el merge?

Angular + NestJS: ejemplo de responsabilidad

Operacion	Donde deberia vivir principalmente
Mostrar lista de productos	Angular: componente y template.
Llamar a GET /api/productos	Angular: servicio con HttpClient.
Validar que el usuario tenga permiso	NestJS: backend/autorizacion.
Calcular total de un pedido	Backend/dominio, no solo frontend.
Guardar productos	Backend + base de datos.

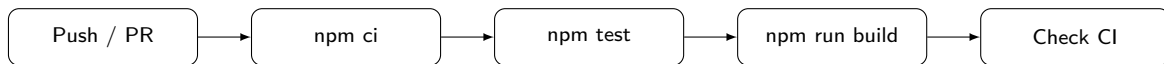
Idea clave

Angular puede validar y mejorar la experiencia, pero la seguridad y las reglas criticas deben aplicarse en el backend.

Angular dentro de CI

Que debe validar el pipeline

Un proyecto Angular puede fallar por dependencias, pruebas, errores TypeScript o problemas de build. CI ayuda a detectarlo antes del merge.



Idea clave

El pipeline no prueba solo que el código exista: verifica que el proyecto pueda instalarse, probarse y construirse en un entorno limpio.

Ejemplo de job CI para Angular

```
frontend:
  runs-on: ubuntu-latest

  defaults:
    run:
      working-directory: frontend

  steps:
    - uses: actions/checkout@v7

    - uses: actions/setup-node@v7
      with:
        node-version: '20'
        cache: 'npm'
        cache-dependency-path: frontend/package-lock.json

    - run: npm ci
    - run: npm test
    - run: npm run build
```

Lectura: el job se ejecuta en ubuntu-latest y los comandos se ejecutan dentro de frontend/.

Actividad: leer una arquitectura Angular

Caso

Un equipo crea un componente `ProductosComponent` que valida permisos, calcula descuentos, llama al backend y almacena datos en `localStorage`.

Preguntas para discutir:

- 1 Que responsabilidades son propias del componente?
- 2 Que deberia moverse a un servicio Angular?
- 3 Que deberia estar en el backend NestJS?
- 4 Que podria probarse con `npm test`?
- 5 Que podria fallar durante `npm run build`?

- 1 El componente deberia coordinar la vista y la interaccion.
- 2 El acceso a datos debe delegarse a un servicio Angular.
- 3 Permisos, reglas criticas y persistencia deben validarse en backend.
- 4 Las pruebas pueden cubrir creacion del componente y comportamiento de servicios.
- 5 El build puede fallar por errores TypeScript, imports incorrectos o rutas mal configuradas.

Conclusion

Una buena arquitectura Angular separa responsabilidades: componente para interfaz, servicio para logica reutilizable y backend para reglas criticas.

Errores frecuentes al comenzar con Angular

- Poner demasiada logica en el componente.
- Repetir llamadas HTTP en varios componentes.
- Confundir validacion visual con seguridad.
- No separar vistas, servicios y modelos.
- No ejecutar pruebas ni build antes de subir cambios.
- Desconocer la carpeta donde se debe ejecutar `npm ci` en un monorepo.

Idea clave

El objetivo no es solo que la interfaz funcione, sino que sea mantenible, testeable e integrable en CI.

- 1 Angular es una tecnología de frontend.
- 2 Su arquitectura principal es basada en componentes.
- 3 Los servicios separan lógica reutilizable y acceso a APIs.
- 4 El router organiza la navegación entre vistas.
- 5 Angular se comunica con NestJS mediante HTTP/JSON.
- 6 En CI, Angular se valida con instalación, pruebas y build.

Pregunta

¿Qué cambia en el pipeline cuando Angular está dentro de una carpeta `frontend/` de un monorepo?

Responder individualmente:

- 1 Que responsabilidad cumple un componente Angular?
- 2 Para que sirve un servicio Angular?
- 3 Que comando construye el proyecto Angular?
- 4 Por que una validacion de seguridad no deberia depender solo del frontend?
- 5 Que deberia ejecutar CI antes de permitir un Pull Request?

Fuentes sugeridas

- Angular Documentation - Overview: <https://angular.dev/overview>
- Angular Documentation - Components: <https://angular.dev/guide/components>
- Angular Documentation - Router: <https://angular.dev/guide/routing>
- Angular Documentation - HTTP Client: <https://angular.dev/guide/http>
- Angular Documentation - CLI: <https://angular.dev/tools/cli>
- Angular Documentation - Version compatibility: <https://angular.dev/reference/versions>